

# 12. Problemi “cammini minimi”



Lezione 24 - @November 17, 2025

## Problemi “Cammini Minimi”

- trovare il cammino minimo tra due vertici
- trovare i cammini tra un vertice  $s$  e tutti gli altri
- trovare i cammini minimo tra ogni coppia di vertici

## Algoritmo di Floyd e Warshall



Cammini minimi tra tutte le coppie di vertici

Supponiamo  $V = \{v_1, v_2, \dots, v_n\}$

$d_{ij}$  = peso (lunghezza) del cammino minimo da  $v_i$  a  $v_j$

**Problema**  $\mathcal{P}$  determinare  $d_{ij}$   $i = 1, \dots, n$  e  $j = 1, \dots, n$

Per  $k = 0, \dots, n$  considero il problema vincolato:

**Problema**  $\mathcal{P}(k)$ : determinare  $d_{ij}^{(k)}$  per  $i = 1, \dots, n$  e  $j = 1, \dots, n$

$d_{ij}^{(k)}$  = peso del cammino minimo da  $v_i$  a  $v_j$  che nei passi intermedi visita esclusivamente vertici di indice  $\leq k$

Allora  $\mathcal{P} = \mathcal{P}(n)$



come risolvere  $\mathcal{P}(k)$

per  $k = 0$  (vertici intermedi)

$$d_{ij}^{(0)} = \begin{cases} \omega(v_i, v_j) & \text{se } \exists (v_i, v_j) \in E, i \neq j \\ 0 & \text{se } v_i = v_j \\ \infty & \text{altrimenti} \end{cases}$$

per  $k > 0$

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

## Pseudo-codice

## ALGORITMO FloydWarshall (Grafo $G$ ) $\rightarrow$ Matrice

Siano  $D_0[1..n, 1..n], \dots, D_n[1..n, 1..n]$  matrici:

```
FOR  $i \leftarrow 1$  TO  $n$  DO
  FOR  $j \leftarrow 1$  TO  $n$  DO
    IF  $i=j$  THEN  $D_0[i,j] \leftarrow 0$ 
    ELSE IF  $(v_i, v_j) \in E$  THEN  $D_0[i,j] \leftarrow \omega(v_i, v_j)$ 
    ELSE  $D_0[i,j] \leftarrow \infty$ 
```

$$d_{ij}^{(0)} = \begin{cases} \omega(v_i, v_j) & \text{se } (v_i, v_j) \in E \text{ e } v_i \neq v_j \\ 0 & \text{se } v_i = v_j \\ \infty & \text{altrimenti} \end{cases}$$

```
FOR  $k \leftarrow 1$  TO  $n$  DO
  FOR  $i \leftarrow 1$  TO  $n$  DO
    FOR  $j \leftarrow 1$  TO  $n$  DO
      IF  $D_{k-1}[i,k] + D_{k-1}[k,j] < D_{k-1}[i,j]$  THEN
         $D_k[i,j] \leftarrow D_{k-1}[i,k] + D_{k-1}[k,j]$ 
      ELSE
         $D_k[i,j] \leftarrow D_{k-1}[i,j]$ 
```

RETURN  $D_n$



Lezione 25 - @November 19, 2025

## Analisi dell'algoritmo

- Tempo  $O(n^3)$
- Spazio  $O(n^3)$

## Ottimizzazione dell'algoritmo

dalla lezione 24

per  $k > 0$ :

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

- per  $j = k$ 
  - Se  $d_{kk}^{(k-1)} = 0$

$$d_{ik}^{(k)} = \min\{d_{ik}^{(k-1)}, d_{ik}^{(k-1)} + d_{kk}^{(k-1)}\} = d_{ik}^{(k-1)}$$

- per  $i = k$ 
  - Se  $d_{kk}^{(k-1)} = 0$

$$d_{ik}^{(k)} = \min\{d_{kj}^{(k-1)}, d_{kk}^{(k-1)} + d_{kj}^{(k-1)}\} = d_{kj}^{(k-1)}$$

$D_k(i, k)$        $D_k(k, j)$   
 "                      "  
 IF  $D_{k-1}[i, k] + D_{k-1}[k, j] < D_{k-1}[i, j]$  THEN  
 $D_k[i, j] \leftarrow D_{k-1}[i, k] + D_{k-1}[k, j]$   
 ELSE  
 $D_k[i, j] \leftarrow D_{k-1}[i, j]$

l'algoritmo diventa:

```

IF D[i,k]+D[k,j] < D[i,j] THEN
    D[i,j] <- D[i,k] + D[k,j]
    
```

ALGORITMO Floyd-Warshall (Grafo  $G$ )  $\rightarrow$  Matrice

Sia  $D[1..n, 1..n]$  una matrice

FOR  $i \leftarrow 1$  TO  $n$  DO

    FOR  $j \leftarrow 1$  TO  $n$  DO

        IF  $i = j$  THEN  $D[i, j] \leftarrow 0$

        ELSE IF  $(v_i, v_j) \in E$  THEN  $D[i, j] \leftarrow \omega(i, j)$

        ELSE  $D[i, j] \leftarrow \infty$

FOR  $k \leftarrow 1$  TO  $n$  DO

    FOR  $i \leftarrow 1$  TO  $n$  DO

        FOR  $j \leftarrow 1$  TO  $n$  DO

            IF  $D[i, k] + D[k, j] < D[i, j]$  THEN

$D[i, j] \leftarrow D[i, k] + D[k, j]$

RETURN  $D$

$d_{ij}^{(0)} = \begin{cases} \omega(v_i, v_j) & \text{se } (v_i, v_j) \in E \text{ e } v_i \neq v_j \\ 0 & \text{se } v_i = v_j \\ \infty & \text{altrimenti} \end{cases}$   
 $d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$

$D$   $\leftrightarrow$   $D_n$

IF  $D_{k-1}[i, k] + D_{k-1}[k, j] < D_{k-1}[i, j]$  THEN  
 $D_k[i, j] \leftarrow D_{k-1}[i, k] + D_{k-1}[k, j]$   
 ELSE  
 $D_k[i, j] \leftarrow D_{k-1}[i, j]$

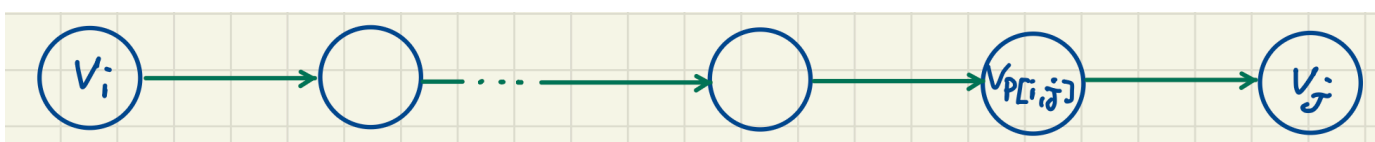
#### Nuova analisi:

- tempo  $O(n^3)$
- spazio  $O(n^2)$

## Come ricavare il cammino minimo tra $v_i$ e $v_j$ ?

Matrice ausiliaria  $P$ :

$P[i, j]$  = indice del penultimo vertice sul cammino da  $v_i$  a  $v_j$  (0 se  $i = j$  o non c'è alcun cammino da  $v_i$  a  $v_j$ )



Dai valori contenuti alla fine in  $P$  si può ricavare il cammino

```
ALGORITMO FloydWarshall (Grafo  $G$ )  $\rightarrow$  Matrice  
Siano  $D[1..n, 1..n]$  una matrice, Sia  $P[1..n, 1..n]$  un array  
FOR  $i \leftarrow 1$  TO  $n$  DO  
  FOR  $j \leftarrow 1$  TO  $n$  DO  
     $P[i, j] \leftarrow 0$   
    IF  $i = j$  THEN  $D[i, j] \leftarrow 0$   
    ELSE IF  $(v_i, v_j) \in E$  THEN  $D[i, j] \leftarrow \omega(i, j)$   
    ELSE  $D[i, j] \leftarrow \infty$   
  END FOR  
   $P[i, j] \leftarrow i$   
END FOR  
FOR  $k \leftarrow 1$  TO  $n$  DO  
  FOR  $i \leftarrow 1$  TO  $n$  DO  
    FOR  $j \leftarrow 1$  TO  $n$  DO  
      IF  $D[i, k] + D[k, j] < D[i, j]$  THEN  
         $D[i, j] \leftarrow D[i, k] + D[k, j]$   
         $P[i, j] \leftarrow P[k, j]$   
      END IF  
    END FOR  
  END FOR  
END FOR  
RETURN  $D$ 
```

$d_{ij}^{(0)} = \begin{cases} \omega(v_i, v_j) & \text{se } (v_i, v_j) \in E \text{ e } v_i \neq v_j \\ 0 & \text{se } v_i = v_j \\ \infty & \text{altrimenti} \end{cases}$

$d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$

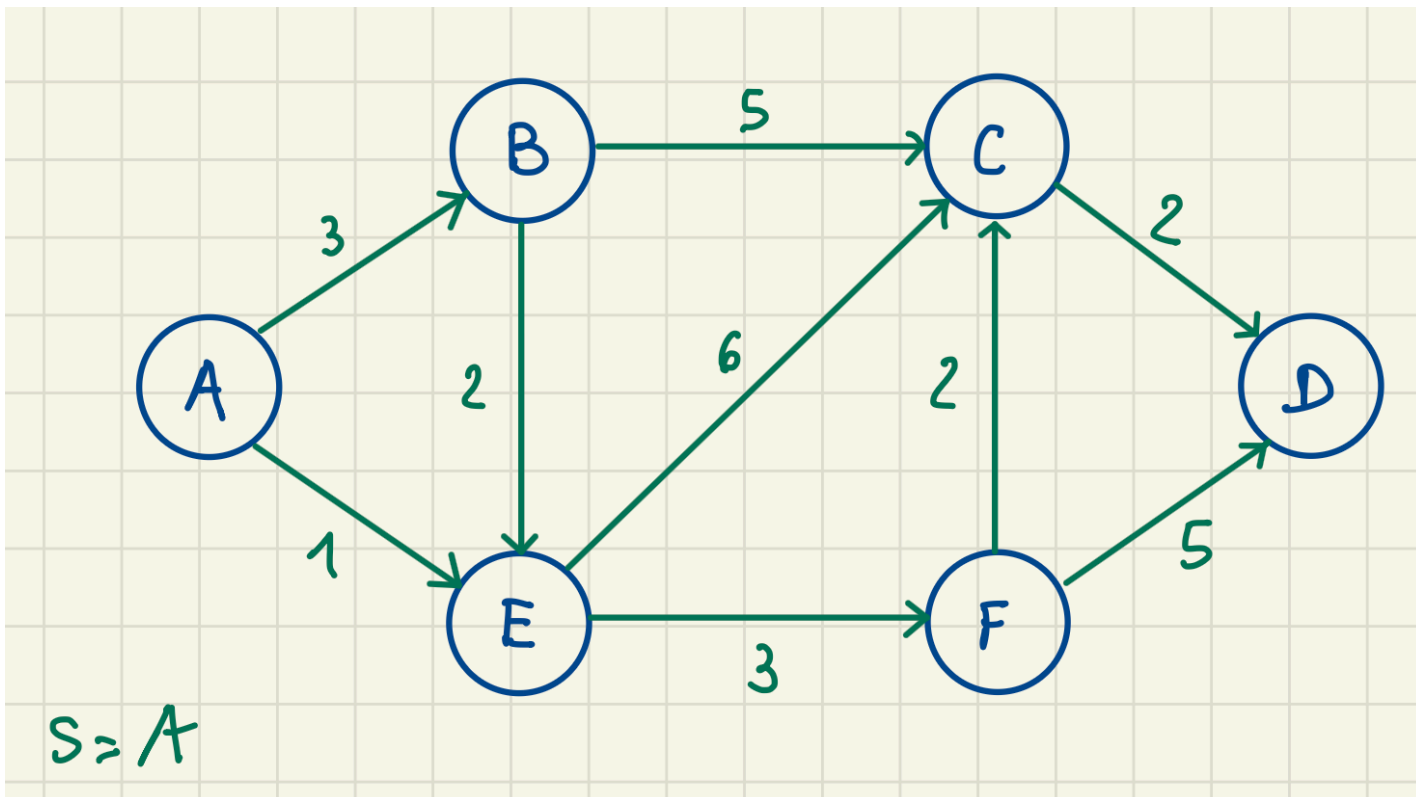


### In caso di pesi negativi?

L'algoritmo Floyd & Warshall funziona anche con grafi contenenti archi con pesi negativi, purché non ci siano cicli di peso negativo

? Cicli negativi  $\rightarrow$  problema mal posto!

### Cammini Minimi da un vertice a tutti gli altri



L'algoritmo di Bellman & Ford

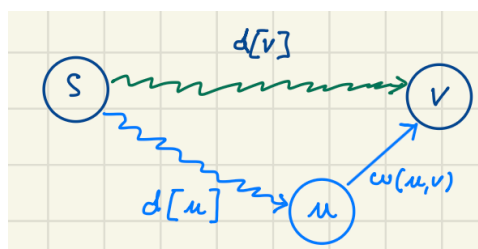
**Input**  $G = (V, E)$   $\omega : E \rightarrow \mathbb{R}$

$s \in V$  vertice di partenza

$d[v]$  = peso del cammino minimo da  $s$  a  $v$  sinora trovato

**inizialmente:**  $d[v] = \begin{cases} 0 & v = s \\ \infty & \text{altrimenti} \end{cases}$

**aggiornamento**



$d[u] + \omega(u, v) < d[v]?$  → ad ogni passo si seguono tutti gli archi del grafo

```
IF  $d[u] + \omega(u, v) < d[v]$  THEN
   $d[v] \leftarrow d[u] + \omega(u, v)$ 
```

**ipotesi**  $G$  pieno di cicli negativi

⇒ tra ogni coppia di vertici esiste un cammino minimo semplice

⇒ è sufficiente considerare cammini composti da al più  $n$  vertici, e dunque  $n - 1$  archi

⇒ In al più  $n - 1$  passi si raggiungono tutti i nodi ottenendo i pesi dei cammini minimi

ALGORITMO Bellman & Ford (Grafo  $G$ , vertice  $s$ )  $\rightarrow$  Vettore

Sia  $d[V]$  un vettore con indici in  $V$

$d[s] \leftarrow 0$

FOR EACH  $v \in V \setminus \{s\}$  DO  $d[v] \leftarrow \infty$

FOR  $k \leftarrow 1$  TO  $n-1$  DO

    FOR EACH  $(u,v) \in E$  DO

        IF  $d[u] + w(u,v) < d[v]$  THEN

$d[v] \leftarrow d[u] + w(u,v)$

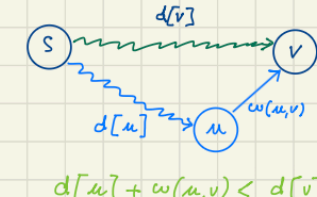
RETURN  $d$

$d[v]$  = peso del cammino minimo da  $s$  a  $v$  sinora trovato

Inizialmente:

$$d[v] = \begin{cases} 0 & \text{se } v=s \\ \infty & \text{altrimenti} \end{cases}$$

Aggiornamento:



$d[u] + w(u,v) < d[v]?$

Lista di archi

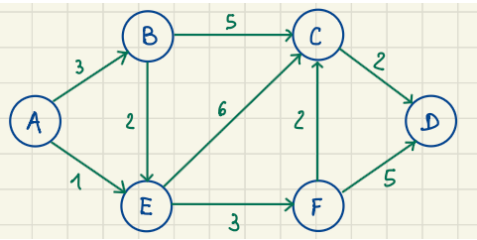
$m = \# E$

$n = \# V$

Tempo  $O(m \cdot n)$

Spazio  $O(n)$

# Algoritmo di Dijkstra



**Input**  $G = (V, E)$   $w : E \rightarrow \mathbb{R}$

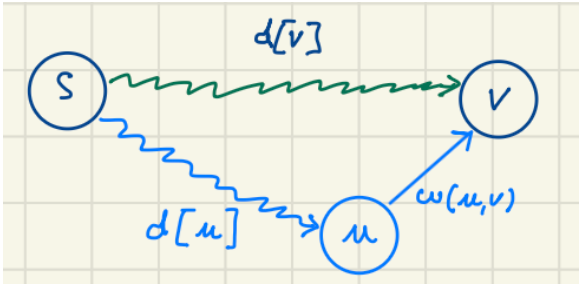
$s \in V$  vertice di partenza

**Ipotesi** pesi non negativi

$d[v]$  = peso del cammino da  $s$  a  $v$  sinora trovato

**Inizialmente**  $d[v] = \begin{cases} 0 & s = v \\ \infty & \text{altrimenti} \end{cases}$

**Aggiornamento** ad ogni passo si considerano gli archi che escono da un vertice scelto con strategia **greedy**



$d[u] + w(u, v) < d[v]?$

IF  $d[u] + w(u, v) < d[v]$  THEN  
 $d[v] \leftarrow d[u] + w(u, v)$

## Implementazione

- grafo rappresentato come lista di adiacenza.
- distanze provvisorie - vettore  $d[v]$

inizialmente  $d[v] = \begin{cases} 0 & v = s \\ \infty & v \neq s \end{cases}$

- $c \subseteq V$  insieme vertici considerati  
inizialmente  $C = V$
- strategia “greedy”  
preleva da  $C$  il vertice  $u$  con  $d[u]$  minima  $d[u]$  diventa definitiva aggiorna  $d[v]$  per ogni  $v$  adiacente a  $u$

```
ALGORITMO Dijkstra(Grafo  $G$ , vertice  $s$ )  $\rightarrow$  Vettore
  Sia  $d[v]$  un vettore di indici in  $V$ 
   $d[s] \leftarrow 0$ 
  FOREACH  $v \in V$  S DO  $d[v] \leftarrow \infty$ 
   $C \leftarrow V$ 
  WHILE  $C \neq \emptyset$  DO
     $u \leftarrow$  elemento di  $C$  con  $d[u]$  minimo
     $C \leftarrow C \setminus \{u\}$ 
    FOREACH  $(u, v) \in E$  DO
      IF  $d[u] + \omega(u, v) < d[v]$  THEN
         $d[v] \leftarrow d[u] + \omega(u, v)$ 
  RETURN  $d$ 
```

teorema di correttezza

dato un grafo  $G = (V, E, \omega)$  e un vertice  $s \in V$ , se  $G$  non contiene archi di peso negativo allora al termine dell’esecuzione dell’algoritmo di Dijkstra per ogni  $v \in V$   $d[v]$  contiene la lunghezza del cammino da  $s$  a  $v$ .

Implementazione Algoritmo di Dijkstra

Conviene rappresentare il grafo con una lista di adiacenza o lista di incidenza.  
Coda con priorità ci permette di trovare il percorso minimo in maniera dinamica.  
Non ha senso usare una coda con priorità per ordinare e basta.  
L’algoritmo **non** si basa su coda a priorità  $\rightarrow$  l’implementazione efficiente si basa su di essa.  
Quando cambia l’array delle distanze, occorre aggiornare anche la coda con i nuovi valori.

Analisi dei Tempi

ipotesi:

- grafo rappresentato come lista di adiacenza
- coda con priorità  $\rightarrow$  min-heap  $\rightarrow$  vettore posizionale.
- $|V| = n, \quad |E| = m$

|    | operazione / costo singolo                                                                        | costo complessivo |
|----|---------------------------------------------------------------------------------------------------|-------------------|
| a  | inizializzazione di $d$                                                                           | $O(n)$            |
| b  | riempimento di $C$                                                                                | $O(n)$            |
| c  | ciclo <code>while</code>                                                                          |                   |
| c1 | <code>deleteMin</code> $O(\log n)$ costo singolo                                                  | $O(n \log n)$     |
| c2 | intestazione <code>foreach</code> $m$ iterazioni complessivamente $\leftarrow$ lista di adiacenza | $O(m)$            |
| c3 | corpo del ciclo $m$ volte <code>changeKey</code> $O(\log n)$                                      | $O(m \log n)$     |

$O(n) + O(n) + O(n \log n) + O(m) + O(m \log n) = O(n \log n) + O(m \log n)$

Se il grafo è connesso:  $m \geq n - 1 \Rightarrow O(m \log n) = \text{tempo}$

Se si dovesse usare l’heap di Fibonacci:

$T(n, m) = O(m + n \log n)$

Come ricavare i cammini minimi?

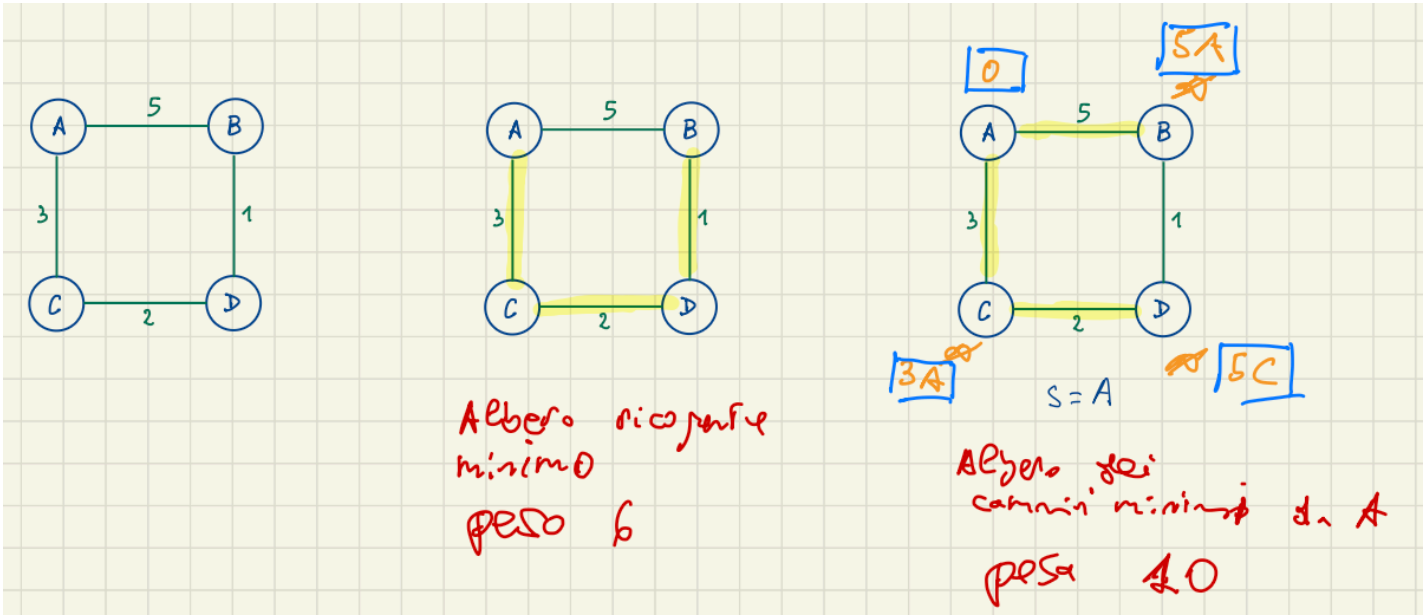
→ introduzione di un nuovo vettore: il vettore dei predecessori `pred[v]` .

```
ALGORITMO Dijkstra(Grafo G, vertice s) → Vettore, Vettore
  Sia d[v] un vettore di indici in V
  Sia pred[v] un vettore di indici in V
  d[s] ← 0
  FOREACH v ∈ V \ {S} DO d[v] ← ∞
  Sia C una coda priorità vuota
  FOREACH v ∈ V DO C.insert(v, d[v])
  WHILE C ≠ ∅ DO
    u ← C.deleteMin()
    IF d[u] + ω(u, v) < d[v] THEN
      d[v] ← d[u] + ω(u, v)
      C.changeKey(v, d[v])
      pred[v] ← u
  RETURN d, pred
```

Si ottiene l’albero dei cammini minimi

Grafi non orientati:

albero ricoprente minimo vs albero dei cammini minimi



sono problemi diversi

Pesi negativi

l’algoritmo di Dijkstra non funziona per grafi con pesi negativi.

Problemi “cammini minimi” ricapitolando

- cammino minimo tra due vertici  
non c’è algoritmo diretto
- cammini minimi da un vertice a tutti gli altri

|       |               |                |
|-------|---------------|----------------|
| Nome  | Dijkstra      | Bellman Ford   |
| Tempo | $O(m \log n)$ | $O(m \cdot n)$ |

| Nome               | Dijkstra                    | Bellman Ford                         |
|--------------------|-----------------------------|--------------------------------------|
| Problema mal posto | per archi con peso negativo | per archi che formano cicli negativi |

- cammini minimi tra ogni coppia di vertici

| Nome               | Floyd-Warshall                       |
|--------------------|--------------------------------------|
| Tempo              | $O(n^3)$                             |
| Problema mal posto | per archi che formano cicli negativi |